Towards Neuromorphic Efficiency: A Comparative Benchmarking of Spiking Neural Networks and Artificial Neural Networks

Aryan Gautam Jadhav
Dept. of Artificial Intelligence and Robotics
Hof University of Applied Sciences
Hof, Germany
jadhavaryan09@gmail.com

***Abstract*—Traditional computer vision systems rely on frame-based processing, where entire images are captured and analyzed at fixed intervals regardless of scene activity. This approach generates significant redundant data, leading to high latency and excessive power consumption that hinders deployment on battery-constrained edge devices. Neuromorphic computing addresses this limitation by processing visual information as sparse, asynchronous events triggered only by changes in the scene. This paper presents a comparative benchmarking of a deep Spiking Neural Network (SNN) against a standard Artificial Neural Network (ANN) on the event-based N-MNIST dataset. The proposed convolutional SNN architecture utilizes Leaky Integrate-and-Fire (LIF) neurons and employs the fast sigmoid surrogate gradient method for training. To ensure a fair comparison of algorithmic efficiency, a structurally identical ANN model serves as the control baseline. Experimental results demonstrate that the SNN achieves a superior classification accuracy of 98.33% (vs. 97.39% for ANN) while exhibiting a temporal sparsity of 96.71%. Based on theoretical energy benchmarks for 45nm CMOS technology, the SNN reduces estimated energy consumption by 59.6% (3,174 nJ vs. 7,864 nJ per inference) compared to the baseline. Furthermore, successful deployment on ARM-based (Apple M2) hardware validates the practical feasibility of supervised spiking neural networks for energy-efficient neuromorphic vision applications.**

***Index Terms*—Spiking Neural Networks (SNN), Artificial Neural Networks (ANN), Convolutional Neural Networks (CNN), Neuromorphic Computing, N-MNIST, Surrogate Gradients, Energy Efficiency.**

I. Introduction

Deep learning has achieved remarkable success in computer vision, enabling high-performance solutions for tasks such as object recognition, autonomous navigation, and medical image analysis. These advances, however, are predominantly based on dense, frame-based processing, in which convolutional neural networks (CNNs) operate on full image frames captured at fixed intervals. This method introduces redundancy, as all pixels are processed regardless of whether meaningful changes occur in the scene or not. For energy-constrained edge devices, including mobile robots, drones, and embedded sensors, such inefficiency leads to increased power consumption and latency, limiting autonomous operation.

Event-based vision offers an alternative approach to visual sensing. Instead of transmitting full image frames, event-based sensors generate asynchronous events only when changes in luminance occur at individual pixel locations [1]. This representation produces sparse spatiotemporal data and reduces redundant computation. The Neuromorphic-MNIST (N-MNIST) dataset provides a widely used benchmark for evaluating learning algorithms on event-based visual data by encoding handwritten digit images as event streams [5].

Spiking Neural Networks (SNNs) are well suited for processing event-based data due to their temporal nature. In contrast to Artificial Neural Networks (ANNs), which rely on continuous-valued activations, SNNs process information using discrete spike events and neuron dynamics that evolve over time. From a computational perspective, this enables sparse synaptic activity and the potential replacement of multiply-accumulate (MAC) operations with simpler accumulate operations (AC), leading to improved energy efficiency [4].

A key challenge in training deep SNNs arises from the non-differentiability of spike generation, which prevents the direct application of conventional backpropagation. Earlier learning approaches, such as spike-timing-dependent plasticity (STDP), were biologically motivated but limited in scalability and performance [6]. Recent developments in surrogate gradient learning address this limitation by approximating spike derivatives [3], enabling supervised training of deep SNNs using gradient-based optimization techniques.

Even with these improvements, it's still hard to fairly compare SNNs and ANNs. Differences in model design and how data is prepared often make it unclear whether the advantages come from spiking behavior or from other factors.

The main contributions of this paper can be summarized as follows:

1) Algorithmic Benchmarking: A controlled comparison is performed in which a convolutional Spiking Neural Network (SNN) with Leaky Integrate-and-Fire (LIF) neurons trained using a Fast Sigmoid surrogate gradient is evaluated against a structurally identical Artificial Neural

Network using ReLU activations on time-collapsed N-MNIST data.
2) Energy Quantification: The energy estimation of the SNN is done using theoretical energy benchmarks for 45 nm CMOS technology, and the energy savings are linked to the sparse timing of spiking activity.
3) Edge Feasibility: The SNN is also evaluated on ARM-based Apple Silicon (M2) hardware, showing that supervised spiking neural networks can run on modern local computing devices.

The remainder of this paper is organized as follows: Section II outlines the theoretical background of LIF neurons and surrogate gradients. Section III details the N-MNIST dataset and network architectures. Section IV presents the experimental results and energy analysis, and Section V concludes the study.

II. Theoretical Background

The operational principles of the models employed in this study contrast the continuous approximations of standard deep learning with the physics of biological neural circuits. This section first outlines the mathematical formulation of the Artificial Neural Network (ANN) baseline, followed by the Spiking Neural Network (SNN) model utilized for temporal processing and the gradient approximation techniques required for supervised learning.

A. Artificial Neural Network Baseline

To establish a performance baseline, a standard Artificial Neural Network (ANN) is employed. As formalized in foundational work on Convolutional Networks [12], the ANN operates by propagating real-valued signals through layers of computation. For a given layer l, the output activation $A^{(l)}$ is computed as in (1): :

$$A^{(l)} = f(W^{(l)}A^{(l-1)} + b^{(l)}), \tag{1}$$

where $W^{(l)}$ is the weight matrix, $b^{(l)}$ is the bias vector, and $f(\cdot)$ is the non-linear activation function.

In this study, the Rectified Linear Unit (ReLU) is utilized as the activation function. Originally proposed to improve training convergence in deep architectures [13], it is defined in (2):

$$f(x) = \max(0, x). \tag{2}$$

Unlike SNNs, which process temporal sequences, the ANN processes static inputs. Consequently, the temporal dimension of the N-MNIST event stream is collapsed via temporal accumulation prior to inference, effectively treating the dynamic event stream as a static intensity image.

B. Spiking Neural Networks

Spiking Neural Networks (SNNs) represent a fundamental departure from the continuous processing paradigm of Artificial Neural Networks (ANNs). While standard ANNs communicate via real-valued activations, SNNs encode information into discrete, asynchronous events known as spikes [2]. This binary mechanism ($\{0, 1\}$) mimics the efficient, event-driven nature of biological brains, where energy is consumed primarily during signal transmission rather than continuously [9]. By processing data only when changes occur, SNNs offer a theoretical advantage in energy efficiency for temporal tasks compared to frame-based deep learning models [4].

C. The Leaky Integrate-and-Fire (LIF) Neuron

The fundamental processing unit utilized in this architecture is the Leaky Integrate-and-Fire (LIF) neuron. Originally formulated in [11], this model provides a computationally efficient approximation of the electrical dynamics across a biological cell membrane.

1) Continuous Dynamics: The neuron is modeled as a circuit consisting of a resistor and a capacitor in parallel [11]. The membrane potential $V(t)$ represents the electrical charge stored by the cell. Left undisturbed, this charge leaks away over time, providing the neuron with a form of short-term memory. The evolution of the potential is governed by the standard RC circuit differential equation (3) [11]:

$$\tau \frac{dV(t)}{dt} = -(V(t) - V_{\text{rest}}) + I(t), \tag{3}$$

where τ is the membrane time constant, V_{rest} is the resting potential, and $I(t)$ is the input current.

2) Discrete Implementation: For implementation in digital simulation, the continuous dynamics are discretized using the Forward Euler method [11]. The update rule (4) for the membrane potential at a discrete time step t is defined as:

$$V[t] = \beta V[t-1] + WX[t] - S[t-1]V_{\text{th}}, \tag{4}$$

This equation dictates the state transitions of the neuron:

- The term $\beta V[t-1]$ represents the decay of the membrane potential. The decay factor is set to $\beta = 0.9$, ensuring that the neuron retains a portion of its previous state.
- $WX[t]$ represents the synaptic input current integrated at the current time step.
- The term $-S[t-1]V_{\text{th}}$ implements a "Soft Reset" mechanism. As detailed in [8], when a spike is generated ($S[t-1] = 1$), the threshold voltage is subtracted from the potential rather than resetting it to zero. This preserves residual information and improves gradient flow during training [8].

3) Spike Generation: The non-linear firing mechanism is defined by the Heaviside step function in (5). A spike $S[t]$ is emitted if the membrane potential exceeds the firing threshold V_{th}:

$$S[t] = \Theta(V[t] - V_{\text{th}}) = \begin{cases} 1, & \text{if } V[t] \geq V_{\text{th}} \\ 0, & \text{if } V[t] < V_{\text{th}} \end{cases}. \tag{5}$$

If the condition is met, a binary 1 is propagated to the next layer; otherwise, the output is 0 [11].

D. Surrogate Gradient Learning

A significant obstacle in training SNNs is the non-differentiable nature of the spike generation function defined in (5). Standard backpropagation requires calculating the gradient of the output with respect to the membrane potential ($\frac{\partial S}{\partial V}$).

However, the derivative of the Heaviside step function is zero everywhere (except at the threshold, where it is undefined), causing gradients to vanish and effectively halting the learning process [3].

To overcome this 'Vanishing Gradients' problem, the Surrogate Gradient method is applied [3]. During the forward pass, the network utilizes the exact Heaviside function to maintain accurate binary spiking dynamics. During the backward pass, the non-differentiable derivative is replaced by a smooth, continuous approximation.

To do this work, the Fast Sigmoid function is employed as the surrogate, as derived in [7]. Its derivative is given in (6):

$$\frac{\partial S}{\partial V} \approx \sigma'(V) = \frac{1}{(1 + k|V - V_{\text{th}}|)^2}, \tag{6}$$

where k is the slope parameter. A value of $k = 25$ is selected to create a narrow, steep gradient window around the threshold. This ensures that weight updates are triggered primarily when the membrane potential is close to the firing limit, thereby stabilizing the training of deep architectures [7]. The surrogate gradient is normalized, with the slope parameter k controlling only the sharpness of the gradient around the firing threshold, while the peak gradient value is fixed to unity. The forward spike generation and the corresponding surrogate gradient approximation used during backpropagation are illustrated in Fig. 1.

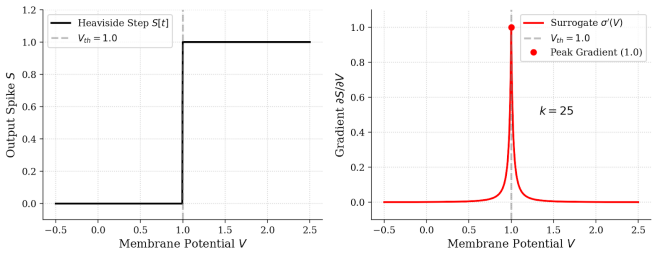

Fig. 1: Forward and backward operations of a spiking neuron with surrogate gradient learning. (a) Spike generation using the Heaviside step function. (b) Fast Sigmoid surrogate gradient used during backpropagation ($k = 25$).

III. Methodology

This study implements a controlled comparative analysis between a frame-based Artificial Neural Network (ANN) and an event-driven Spiking Neural Network (SNN). To ensure the validity of the comparison, both networks share an identical architecture, differing only in their neuron models and data ingestion mechanisms. The experimental framework is designed to isolate the impact of spiking dynamics on performance and computational efficiency.

A. Data Ingestion and Preprocessing

The Neuromorphic-MNIST (N-MNIST) dataset [5] serves as the experimental benchmark. Unlike static images, N-MNIST data consists of asynchronous streams of discrete events (spikes) generated by a Dynamic Vision Sensor (DVS). Processing this data requires two distinct ingestion strategies to suit the operational principles of each network type.

1) SNN Temporal Ingestion: For the Spiking Neural Network, preserving the time-domain information is critical. The Tonic neuromorphic library is utilized to handle event-based processing. The raw event stream is transformed into a sequence of frames using the ToFrame transformation method [10]. Raw asynchronous events are difficult to process directly in batch-based deep learning frameworks. By binning events into discrete time steps, a compatible tensor structure of (Time $\times$ Batch $\times$ Channels $\times$ Height $\times$ Width) is created while retaining the temporal dynamics required for spiking neurons [8]. A temporal window of 15 time steps ($T = 15$) was selected. This duration was chosen as a balance: it gives the neurons enough time to process and distinguish between classes, while avoiding the extra computation cost of longer simulation windows. Fig. 2 shows a representative input sample at particular time steps generated from the N-MNIST dataset using the Python preprocessing and visualization code developed in this work. The white pixels correspond to ON polarity and black to OFF polarity.

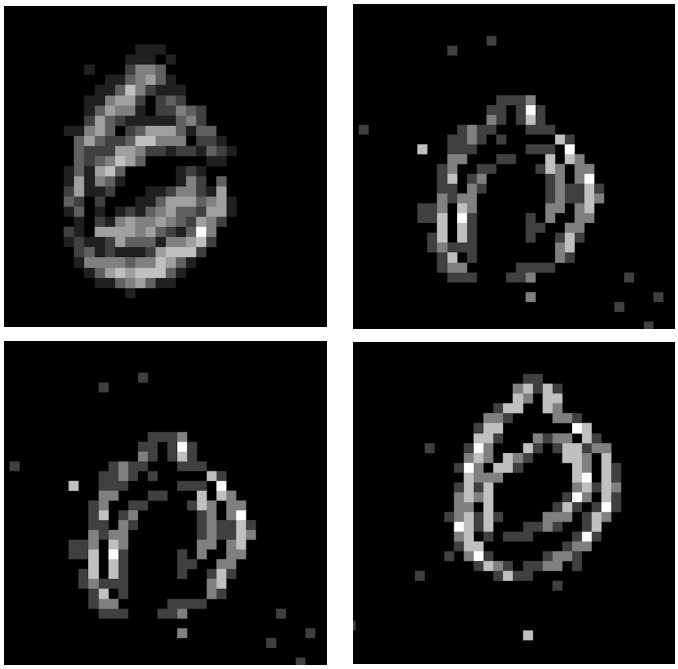

Fig. 2: SNN input of digit 0 (a) At $T = 2$ (b) At $T = 4$ (c) At $T = 8$ (d) At $T = 12$.

2) ANN Static Ingestion: For the Artificial Neural Network baseline, the temporal dimension is collapsed to fit the network's static nature. A summation strategy was implemented where the event stream is aggregated across all time steps into a single intensity frame. Mathematically, the input tensor is summed along the time axis, reducing the dimensions from $(T \times C \times H \times W)$ to $(C \times H \times W)$. This effectively mimics a long-exposure photograph, converting the dynamic event stream into a static image that standard Convolutional Neural Networks can process [14]. Fig 3 shows a representative time-

collapsed input generated from the N-MNIST dataset using the preprocessing pipeline implemented in this work. Colored pixels indicate the spatial locations of accumulated events across the temporal window, which are processed by the ANN as a static image.

Fig. 3: Time-collapsed N-MNIST input for ANN of digit 7.

B. Network Architecture Design

To isolate the efficiency gains strictly to spiking dynamics, both networks share a specific Convolutional Neural Network (CNN) topology: two convolutional layers, two max-pooling layers, and a final fully connected classifier.

1) Common Architecture: The architecture begins with a convolutional layer consisting of 12 filters with a 5×5 kernel size. This is followed by a 2×2 max-pooling operation to reduce spatial dimensionality. The second convolutional layer expands the feature space to 32 filters, again using a 5×5 kernel. This specific filter count of 32 was chosen to produce a flattened feature vector of exactly 800 units (32 channels $\times$ 5×5 pixels), which provides a dense but manageable input for the final classification layer. The major difference here is that the ANN uses ReLU activation after each convolutional layer while SNN uses a LIF neuron.

2) SNN-Specific Component Design: The SNN replaces standard activations with Leaky Integrate-and-Fire (LIF) neurons. The design choices for these neurons are specific to the requirements of the N-MNIST task:

- Neuron Model: The snn.Leaky model from the snnTorch framework is employed as LIF neuron. This model introduces a memory state (membrane potential) that persists across time steps [8].
- Decay Rate (β): The decay rate is set to 0.9. A beta value of 0.9 implies the neuron retains 90% of its voltage from the previous step. This high retention is crucial for N-MNIST because the input events can be sparse; the neuron is required to remember previous inputs long enough to accumulate sufficient charge to fire.
- Surrogate Gradient: Training SNNs is hindered by the non-differentiable nature of spike generation (a step function). This was addressed by injecting a *Fast Sigmoid* surrogate gradient during the backward pass. The slope of the sigmoid is set to $k = 25$. This creates a very steep curve, closely approximating the true binary behavior of a spike while still providing a valid gradient for the optimizer to update weights [3].

C. Training Protocol and Optimization

The training framework was designed to handle the discrete behavior of spiking neurons while still allowing gradient-based learning. This is achieved using Backpropagation Through Time (BPTT) together with surrogate gradient learning, as described in [3][7][8].

1) Initialization: Both SNN and ANN followed the weight and state initializations as follows:

- Weight Initialization: Synaptic weights for both the ANN and SNN were initialized using the default PyTorch configuration [8], which prevents vanishing gradients during the initial pass.
- Membrane State Reset: In the SNN, the internal state (membrane potential) of the LIF neurons persists across time steps. To prevent historical activity from corrupting subsequent predictions, the membrane state is reset to zero at the beginning of each training batch.

2) SNN Training Strategy: During the forward pass, the input tensor is processed sequentially over 15 discrete time steps. At each layer, the incoming data is convolved to produce a synaptic current, which is directly integrated into the membrane potential of the LIF neurons. When this potential exceeds a fixed threshold, the neuron generates a binary spike (1) and resets its membrane voltage; otherwise, it outputs a zero (0). These generated spikes are recorded at each layer. The final classification is derived by aggregating the total number of spikes emitted by the output neurons over the full simulation window, where the class with the highest accumulated count is selected as the prediction. Major concepts that allow this are explained below:

Backpropagation Through Time (BPTT): To enable learning, the SNN is unrolled over a temporal window of 15 time steps. This creates a computational graph where the weights are shared across all time steps. During the forward pass, the membrane potential is updated iteratively. During the backward pass, gradients are propagated through time to update the synaptic weights [8].

Surrogate Gradient Learning: A primary challenge in training SNNs is the non-differentiable nature of the spike generation function, which blocks gradient flow (vanishing gradients). To overcome this, the Fast Sigmoid function is injected as a surrogate gradient during the backward pass, as formally defined in Section II.

Loss Function and Encoding: The network utilizes a rate-coding scheme, where the class prediction is determined by the frequency of spiking activity. To align the optimization with this objective, the Cross-Entropy Spike Count Loss was employed. Instead of calculating error on a single value, this loss function accumulates the output spikes from the

final layer over the full 15 time steps. It encourages the neuron corresponding to the correct class to maximize its total firing count, effectively training the network to output a high frequency of spikes for the target digit [8].

Convergence: The SNN was trained for 10 epochs. It requires temporal credit assignment through BPTT, which is more complex than the static spatial mapping of the ANN, thus necessitating a longer training duration to reach optimal convergence.

3) ANN Training Strategy: The ANN baseline served as a static control and was trained for 5 epochs. The ANN converges faster due to its static input representation. Extending training beyond this point was found to produce diminishing results.

Loss Function: The ANN employed the standard Cross-Entropy Loss. This function calculates the error based on the softmax probability distribution of the output class scores, which is the standard methodology for static image classification tasks [14].

4) Optimization Configuration: Optimization for both architectures was performed using the Adam optimizer with a learning rate of 0.001 (1×10^{-3}) and a batch size of 128. This configuration was selected to ensure stable convergence given the sparse nature of the event data.

D. Evaluation Framework

To quantify the trade-offs between the static and spiking models, a specialized evaluation routine was developed to extract internal network metrics that are not available during standard training.

1) Sparsity Quantification Logic: To strictly quantify the network redundancy, custom evaluation functions were implemented to intercept intermediate layer outputs during inference. The sparsity logic for both (7) and (8) is derived from the standard definition of activity density, serving as a direct multiplier for the energy efficiency calculations.

ANN Sparsity (ReLU Inactivity): For the ANN, sparsity is a consequence of the Rectified Linear Unit (ReLU) activation function, which zeros negative inputs. A custom function was implemented to count the exact number of zero-valued elements in the feature maps. The sparsity percentage is calculated using (7):

$$\text{Sparsity}_{\text{ANN}} = \left(\frac{\sum (ReLU_{out} == 0)}{N_{\text{total}}} \right) \times 100 \qquad (7)$$

where N_{total} represents the total number of neurons in the analyzed layers. The calculation done in (7) is defined in Appendix C.

SNN Sparsity (Temporal Silence): For the SNN, sparsity is defined as the absence of spiking activity over time. The evaluation script aggregates the total number of spikes S_{total} emitted by the monitored spiking neurons over the full temporal window of $T = 15$ time steps. This value is normalized by the maximum possible number of spikes, given by the product of the batch size, number of time steps, and the number of neurons. Sparsity is then computed by:

$$\text{Sparsity}_{\text{SNN}} = \left(1 - \frac{S_{\text{total}}}{\text{Batch} \times T \times N_{\text{neurons}}} \right) \times 100 \qquad (8)$$

This formula explicitly quantifies the percentage of time steps where neurons remain silent by subtracting the activity from total capacity as shown in (8). The metrics are simplified in Appendix C.

2) Theoretical Energy Estimation: Since direct hardware power measurement was not possible, theoretical energy consumption was estimated using operation counts, a standard benchmarking practice. The energy cost is derived strictly from the operations required to process a single inference [4]. Even though these values are derived for 45nm hardware their validity for this use case is defined in Appendix A.

ANN Energy (MACs): The energy for the ANN is static and determined by the network topology. The total number of Multiply-Accumulate (MAC) operations was derived analytically. Each MAC operation is assigned a theoretical energy cost of 4.6 pJ (32-bit floating point) [4].

$$E_{\text{ANN}} = \text{Total MACs} \times 4.6 \text{ pJ} \qquad (9)$$

SNN Energy (SOPs): The energy for the SNN is dynamic and data-dependent. Unlike the ANN, energy is consumed only when a spike occurs. The evaluation script tracks the precise number of spikes emitted by each layer and multiplies this by the fan-out (connections to the next layer) to determine the total Synaptic Operations (SOPs). Because SNNs operate via accumulation rather than multiplication, each SOP is assigned a lower energy cost of 0.9 pJ [4].

$$E_{\text{SNN}} = \sum (\text{Spikes} \times \text{Fan-Out}) \times 0.9 \text{ pJ} \qquad (10)$$

This formulation highlights the fundamental efficiency advantage of SNNs: if no spike is generated, the theoretical energy cost for that connection is zero. Both (9) and (10) are simplified in Appendix C.

IV. Experimental Results

This section evaluates the performance of the proposed neuromorphic architecture against the static baseline. The experiments compare a standard Artificial Neural Network (ANN) against two variations of the Spiking Neural Network (SNN) trained on different hardware backends (NVIDIA CUDA vs. Apple MPS) to observe how this affects sparsity and energy characteristics.

Table I summarizes the key performance metrics across all three configurations.

A. Classification Accuracy

As observed in Table I, both SNN variants outperformed the ANN baseline. The CUDA SNN achieved the highest accuracy of 98.33%, surpassing the ANN (97.39%) by approximately 0.94%. This result indicates that, for the N-MNIST task, spiking computation does not necessarily incur an accuracy penalty [3]. The performance gain is attributed to the temporal

TABLE I: Performance Benchmarking

Metric	ANN (Baseline)	SNN (Proposed)	
		CUDA	*MPS*
Accuracy	97.39%	**98.33%**	98.17%
Average Sparsity	57.19%	96.71%	**97.72%**
Theoretical Energy / Image	7,864 nJ	3,174 nJ	**2,477 nJ**

depth of the SNN ($T = 15$). While the ANN is forced to classify based on a spatially collapsed long-exposure frame as explained in Section III-A. The confusion matrices for both ANN and SNN can be seen in Appendix D.

B. Sparsity Analysis

The most distinct divergence between the architectures lies in network activity (Sparsity), as detailed in the second row of Table I.

1) Deterministic vs. Temporal Sparsity: The ANN exhibits a fixed sparsity range ($\approx 57\%$), which is strictly deterministic. This arises solely from the ReLU activation function zeroing out negative feature maps. In contrast, the SNNs demonstrate massive temporal sparsity ($> 96\%$). This is due to the Leaky Integrate-and-Fire (LIF) neuron model, which remains silent (output logic 0) until the membrane potential reaches a threshold [3][15].

2) Variation: Interestingly, the SNN trained on the MPS backend converged with 97.72% sparsity, higher than the CUDA SNN (96.71%). This suggests that backend-dependent numerical effects, such as floating-point precision and execution order, can influence the learned spike activity patterns [16]. The details of the exact calculation can be found in Appendix C. Fig. 4 presents a spike raster plot of the SNN during inference, demonstrating that most neurons remain inactive for the majority of the simulation window via the white space in background, consistent with the measured sparsity exceeding 96%. Refer to Fig. D3 and D4 in Appendix D for supplementary plots.

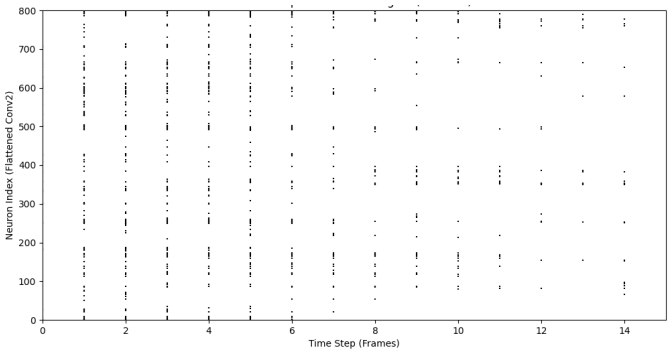

Fig. 4: Spike raster plot for digit 7.

C. Energy Efficiency

The energy values in Table I were calculated using the standard theoretical operation-counting method [4].

1) ANN Energy (Fixed): The ANN energy consumption is constant at $7,864$ nJ per image. Because standard ANNs perform matrix multiplications across the entire feature map regardless of content, their cost is fixed by the topology (1.7×10^6 MACs) The detailed calculation can be found in Appendix C.

2) SNN Energy (Dynamic): The SNN energy is dynamic and directly proportional to the spike count.

- CUDA SNN: Estimates $3,174$ nJ per image, a $2.48\times$ improvement over the baseline.
- MPS SNN: Estimates only $2,477$ nJ per image, a $3.17\times$ improvement.

The reduction in energy between the two SNN models (from $3,174$ to $2,477$ nJ) directly mirrors the increase in sparsity (from 96.71% to 97.72%). This result supports the view that neuromorphic energy efficiency, is a data-dependent property: the fewer spikes the network generates, the less energy it consumes [3].

V. Conclusion and Future Work

A controlled benchmarking study was presented comparing Spiking Neural Networks (SNNs) with Artificial Neural Networks (ANNs) on the N-MNIST neuromorphic dataset. A custom evaluation framework was implemented on modern hardware accelerators (NVIDIA CUDA and Apple MPS) to quantify the trade-offs between classification accuracy, temporal sparsity, and energy efficiency.

The experimental results demonstrate that the proposed SNN architecture matches and slightly surpasses the static baseline in classification performance, achieving a peak accuracy of 98.33% compared to 97.39% for the ANN. This 0.94% improvement indicates that the temporal integration capabilities of LIF neurons enable more effective capture of dynamic features in event-stream data than static frame accumulation.

In addition, a theoretical energy estimation based on operation counting was performed. The analysis shows that SNNs are significantly more energy-efficient, consuming up to $3.17\times$ less energy (2,477 nJ vs. 7,864 nJ per inference). This efficiency is directly driven by the high temporal sparsity of the spiking model, in which neurons remain silent for over 97.7% of the simulation steps.

It was also observed that backend-dependent numerical effects during training (MPS vs. CUDA) can lead to variations in sparsity, suggesting that hardware-aware optimization may represent a promising direction for further efficiency improvements.

Future work may investigate deployment of the trained models on dedicated neuromorphic hardware platforms, such as Intel Loihi or SpiNNaker, to validate the theoretical energy estimates using direct power measurements. In addition, extending the proposed architecture to more complex event-based datasets, such as DVS-Gesture, would enable further evaluation of scalability and generalization. Exploring hardware-aware training and inference strategies may also

provide deeper insights into maximizing sparsity and energy efficiency across different computing backends.

REFERENCES

[1] P. Lichtsteiner, C. Posch, and T. Delbruck, "A 128× 128 120 dB 15 μs latency asynchronous temporal contrast vision sensor," *IEEE Journal of Solid-State Circuits*, vol. 43, no. 2, pp. 566–576, 2008.

[2] C. Mead, "Neuromorphic electronic systems," *Proceedings of the IEEE*, vol. 78, no. 10, pp. 1629–1636, 1990.

[3] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 61–69, 2019.

[4] M. Horowitz, "1.1 Computing's energy problem (and what we can do about it)," in *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, pp. 10–14, 2014.

[5] G. Orchard, A. Jayawant, G. K. Cohen, and N. Thakor, "Converting static image datasets to spiking neuromorphic datasets using saccades," *Frontiers in Neuroscience*, vol. 9, p. 437, 2015.

[6] P. U. Diehl and M. Cook, "Unsupervised learning of digit recognition using spike-timing-dependent plasticity," *Frontiers in Computational Neuroscience*, vol. 9, p. 99, 2015.

[7] F. Zenke and S. Ganguli, "SuperSpike: Supervised Learning in Multilayer Spiking Neural Networks," *Neural Computation*, vol. 30, no. 6, pp. 1514–1541, 2018.

[8] J. K. Eshraghian *et al.*, "Training Spiking Neural Networks Using Lessons From Deep Learning," *Proceedings of the IEEE*, vol. 111, no. 9, pp. 1016–1060, 2023.

[9] M. Davies *et al.*, "Loihi: A Neuromorphic Manycore Processor with On-Chip Learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.

[10] G. Lenz, S. Sheik, and G. Orchard, "Tonic: A library for event-based datasets and transformations," in *Proceedings of the International Conference on Neuromorphic Systems 2021*, pp. 1–8, 2021.

[11] L. Lapicque, "Recherches quantitatives sur l'excitation électrique des nerfs traitée comme une polarisation," *Journal de Physiologie et de Pathologie Générale*, vol. 9, pp. 620–635, 1907.

[12] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.

[13] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in Proceedings of the 27th International Conference on Machine Learning (ICML-10), pp. 807–814, 2010.

[14] A. I. Maqueda, A. Loquercio, G. Gallego, N. Garcia, and D. Scaramuzza, "Event-based vision meets deep learning on steering prediction for self-driving cars," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 5419–5427, 2018.

[15] W. Maass, "Networks of spiking neurons: The third generation of neural network models," Neural Networks, vol. 10, no. 9, pp. 1659–1671, 1997.

[16] J. Gawlikowski et al., "A survey of uncertainty in deep neural networks," arXiv, arXiv:2107.03342, 2021.

[17] N. Rathi and K. Roy, "DIET-SNN: A low-latency spiking neural network with direct input encoding and leakage and threshold optimization," IEEE Transactions on Neural Networks and Learning Systems, vol. 32, no. 12, pp. 5469–5479, Dec. 2021.

APPENDIX A
ENERGY ESTIMATION VALIDITY AND HARDWARE SCALING

The energy consumption values reported in Section IV were derived using the theoretical baselines established by [4], which characterize the energy cost of operations (MACs and ACs) for 45nm CMOS technology. It is acknowledged that the physical hardware used for these experiments—the Apple M2 (MPS) and NVIDIA CUDA GPUs—utilizes significantly more advanced process nodes (5nm). Consequently, the absolute energy values reported in our theoretical analysis are higher than what would be measured on these state-of-the-art chips. However, this theoretical framework remains the accepted standard for algorithmic benchmarking in neuromorphic literature [8][17] for two key reasons:

1) Hardware Independence: This method measures the computational cost of the algorithm itself, rather than the efficiency of a specific chip. By counting the mathematical operations, we create a fair, hardware-agnostic comparison that isolates the architectural benefits of the SNN from the specific power management features of the Apple or NVIDIA hardware.
2) Consistent Efficiency Ratio: While modern chips are more efficient overall, the relative difference in cost between a multiplication (used in ANNs) and a simple addition (used in SNNs) remains consistent across technology nodes. Research indicates that as technology scales down from 45nm to 5nm, the efficiency ratio (the gap between ANN and SNN energy) remains stable, making these estimates a valid predictor of relative performance gains on modern silicon [17].

Thus, the reported values serve as a reliable comparative index to demonstrate the fundamental efficiency of the spiking approach.

APPENDIX B
STATEMENT OF GENERATIVE AI USAGE

In the interest of transparency and research integrity, the author discloses the use of Large Language Models (LLMs) to assist in specific stages of this work. The primary model used was Google Gemini 3 Pro. To avoid repetitive mention of the same text, it is clarified that the LLM was supplied with all the relevant documentations and information as stated in Table II.

The usage was limited to the following:

A. Code and Visualization Support

AI prompts were utilized to generate LaTeX code for formatting tables and creating placeholder structures for figures.

- *Prompt:* "Generate IEEE style LaTeX table code to compare accuracy, sparsity, and energy for ANN and SNN models."
- *Prompt:* "Generate IEEE style LaTeX code to place an image exactly below it's text."

All generated code was manually reviewed, executed, and verified against the experimental data.

TABLE II: AI Prompt Specification

Category	Details
Context	I am a Master's degree student in Artificial Intelligence and Robotics, I have opted for an AI project on topic 'Neuromorphic AI: Spiking Neural Networks'. I have trained the SNN model built via snnTorch framework on N-MNIST dataset using surrogate gradients and BPTT. I have also trained an ANN with same architecture and dataset for benchmarking.
Uploads	I have uploaded my presentation, code files, documentations and referred literature for your reference.
Role	You have to think and act as a computer science and AI professor with over 30 years of experience.
Boundaries	Do not generate any content by yourself. I have to write the contents by myself. Do not hallucinate, answer only using the uploaded materials.
Goal	I am writing an IEEE standard paper using all of these uploaded materials, help me clarify my doubts and refine content regarding the project and it's documentation.

B. Drafting and Summarization

AI assistance was used to refine the clarity of the abstract and summarize related literature. An abstract was drafted originally by the author and uploaded as a pdf to the AI and asked for refinement.

- *Prompt:* "Refine the abstract text provided, do not change the facts and numbers just refine it for academic purpose."
- *Prompt:* "Suggest relevant literature for Spiking Neural Network energy estimation methodology."

The final text, including the abstract and technical descriptions, was critically reviewed and edited by the author to ensure accuracy and alignment with the experimental findings.

APPENDIX C
ENERGY AND SPARSITY MEASUREMENT DETAILS

This appendix documents the exact counting procedures and numeric values used in the evaluation scripts to compute sparsity and theoretical energy metrics for both the Artificial Neural Network (ANN) and the Spiking Neural Network (SNN).

A. ANN Sparsity Measurement (ReLU-Based)

For the ANN baseline, sparsity is measured based on the inactivity of ReLU neurons during inference.

1) Identification of Inactive Neurons: In the ANN model, ReLU is applied after each convolutional layer. Any activation with a negative or zero value is treated as an inactive neuron. During the evaluation pass, the exact number of zero-valued elements in the feature maps is counted explicitly.

2) Exclusion of Output Layer: The final Fully Connected output layer is excluded from the ANN sparsity calculation. In standard ANN architectures, the output layer produces dense logits (raw class scores) which can be negative or positive. Since no activation function zeroes out these values before classification, they do not contribute to algorithmic sparsity.

3) Total Neuron Count: The total number of ANN neurons considered for the sparsity calculation includes all ReLU-activated neurons in the feature extraction layers. Based on the output shapes defined in the code:

- Convolutional Layer 1: 12 feature maps with an output spatial resolution of 30×30, resulting in $12 \times 30 \times 30 = 10,800$ neurons.
- Convolutional Layer 2: 32 feature maps with an output spatial resolution of 11×11, resulting in $32 \times 11 \times 11 = 3,872$ neurons.

The total number of ANN neurons evaluated per sample is therefore $14,672$. The total denominator for the percentage calculation is $14,672 \times$ Batch Size (via PyTorch numel).

4) Sparsity Calculation: For each batch, the total count of zero-valued activations is normalized by the total neuron count. This procedure results in an average ANN sparsity of approximately 57.19%.

B. SNN Sparsity Measurement (Spike-Based)

For the SNN, sparsity is defined as the temporal silence of neurons over the simulation window.

1) Inclusion of Output Layer: Unlike the ANN, the SNN output layer consists of Leaky Integrate-and-Fire (LIF) neurons. These neurons must emit discrete spikes to indicate a class prediction. A silent output neuron (non-firing) consumes no synaptic energy in neuromorphic hardware, therefore the output layer is included in the SNN sparsity calculation.

2) Spike Recording and Capacity: The SNN is simulated for $T = 15$ discrete time steps. After each step, the binary spike outputs are aggregated from all layers. The total capacity is calculated as:

$$C_{\text{total}} = \text{Batch} \times T \times N_{\text{neurons}}$$

where $N_{\text{neurons}} = 14{,}682$ (including the 10 output neurons). The inclusion of the temporal dimension T reflects the fact that sparsity in SNNs is measured over time rather than across a single static activation.

3) Sparsity Calculation: Sparsity is calculated as the percentage of the total capacity that remained silent:

$$\text{Sparsity} = \left(1 - \frac{\text{Total Spikes Observed}}{C_{\text{total}}}\right) \times 100$$

This method yielded sparsity values of 96.71% (CUDA) and 97.72% (MPS).

C. ANN Energy Estimation (MAC-Based)

ANN energy consumption is estimated using hard-coded multiply-accumulate (MAC) operation counts derived from the network topology. The calculation strictly follows the tensor dimensions defined in the evaluation script.

1) Layer-Wise MAC Counting: The number of multiply accumulate (MAC) operations for each convolutional layer is computed as:

$$\text{MACs} = N_{\text{pixels}} \times N_{\text{out}} \times (N_{\text{in}} \times K \times K), \qquad (11)$$

where N_{pixels} denotes the number of spatial output pixels, N_{out} the number of output channels, N_{in} the number of input channels, and K the kernel size. This is the general mathematical formula.

Using (11) The layer-wise breakdown implemented in the code is as follows:

- Convolutional Layer 1: Input $(2, 34, 34)$, output $(12, 30, 30)$ with a 5×5 kernel. The resulting operation count is:
$$30 \times 30 \times 12 \times (2 \times 5 \times 5) = 540{,}000 \text{ MACs}.$$
- Convolutional Layer 2: Input $(12, 15, 15)$, output $(32, 11, 11)$ with a 5×5 kernel. The resulting operation count is:
$$11 \times 11 \times 32 \times (12 \times 5 \times 5) = 1{,}161{,}600 \text{ MACs}.$$
- Fully Connected Layer: Input size 800, output size 10. The resulting operation count is:
$$800 \times 10 = 8{,}000 \text{ MACs}.$$
- Total:
$$540{,}000 + 1{,}161{,}600 + 8{,}000 = 1{,}709{,}600 \text{ MACs}.$$

2) Total ANN Energy Calculation: First, the total operational load is calculated by summing the MACs from all layers:

$$\text{Total MACs} = 1{,}709{,}600 \text{ MACs}$$

Next, using (9) to estimate total energy:

$$\begin{aligned} E_{\text{ANN}} &= \text{Total MACs} \times 4.6 \text{ pJ} \\ &= 1{,}709{,}600 \times 4.6 \times 10^{-12} \text{ J} \\ &= 7{,}864{,}160 \times 10^{-12} \text{ J} \\ &= 7.86416 \times 10^{-6} \text{ J} \\ &\approx 7{,}864 \text{ nJ/image}. \end{aligned}$$

This theoretical value of $7,864$ nJ is constant for every inference pass, as the ANN topology does not change based on input data.

D. SNN Energy Estimation (Spike-Driven)

SNN energy is dynamic and calculated by tracking Synaptic Operations (SOPs).

1) Energy Accounting Logic: Energy is consumed only when a spike is generated. The evaluation script records the spikes generated at each layer and calculates SOPs by multiplying the spike count by the fan-out (connections to the next layer):

- Input $\rightarrow$ L1: Input Spikes $\times$ 300 ($12 \times 5 \times 5$ kernel).
- L1 $\rightarrow$ L2: L1 Output Spikes $\times$ 800 ($32 \times 5 \times 5$ kernel).
- L2 $\rightarrow$ L3: L2 Output Spikes $\times$ 10 (Linear weights).

2) Total SNN Energy: The total SOP count is multiplied by 0.9 pJ (the cost of a floating-point addition). Because the number of spikes varies with input data and hardware training dynamics, the final energy estimation varies, resulting in averages of $3,174$ nJ (CUDA) and $2,477$ nJ (MPS).

APPENDIX D
VISUAL EVIDENCE

This section contains the confusion matrices to support the accuracy claims. Spike raster plots for individual digits are also included here.

A. *Confusion matrices*

To keep the visual contrast ANN is assigned with red colour and SNN with Blue colour as seen in Fig. D1 and Fig. D2. These confusion matrices support the results mentioned in the Section IV Table I. As seen in Fig. D1 and Fig. D2 the diagonal classifications are the maximum number indicating that most of the classifications are correct supporting the above 97% accuracies.

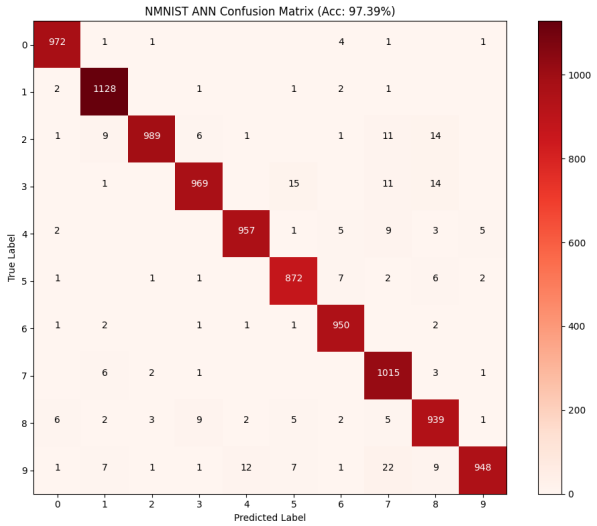

Fig. D1: Confusion Matrix ANN.

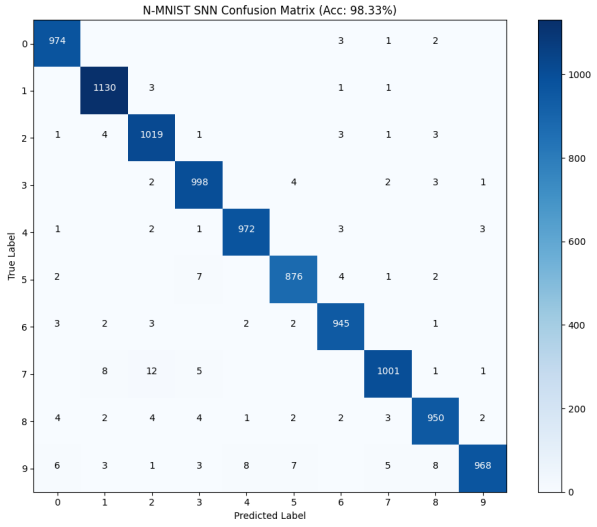

Fig. D2: Confusion Matrix SNN.

B. *SNN Raster plots*

The spike raster plots are a standard way to display and verify SNN sparsity. To understand these plots visually, the black dots seen at each time step from 1 to 15 are the actual spikes generated and the remaining white space corresponds to network silence. This validates the sparsity results in Section IV.

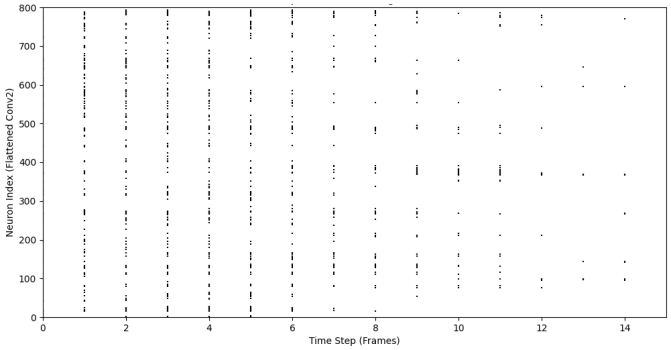

Fig. D3: Spike Raster plot for digit 2.

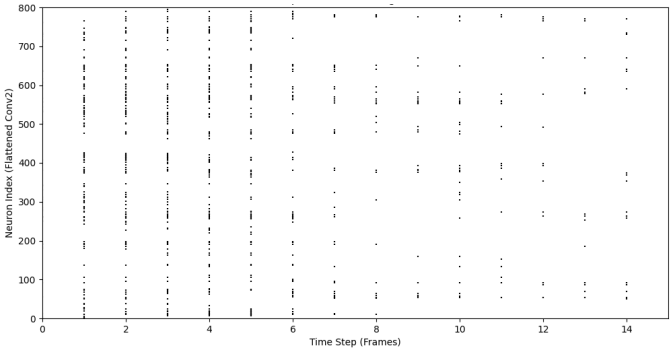

Fig. D4: Spike Raster plot for digit 5.

APPENDIX E
PROJECT PLAN

The project followed a structured development timeline of 15 weeks. During the research phase, two significant methodological changes were made to align the project with neuromorphic standards.

First, the dataset was upgraded from standard MNIST to N-MNIST [5]. Initial experiments revealed that converting static MNIST into Poisson spike trains failed to produce temporal sparsity, as the data lacked time dynamics. N-MNIST was adopted to validate the network's efficiency on real event streams.

Second, the learning framework was shifted from STDP (via Brian2) [6] to supervised Surrogate Gradient Learning using the snnTorch framework [8]. While STDP is biological, recent literature demonstrates that surrogate gradients are currently the most advanced method for training deep Spiking Neural Networks [3].Table III provides the original 15-week execution roadmap, while Table IV summarizes the specific methodological deviations and their justifications.

TABLE III: Project Execution Roadmap (Original 15-Week Plan)

Phase	Timeline	Key Deliverables & Activities
I. Research & Setup	Weeks 1–3 (Oct 15 – Nov 4)	Literature review on SNNs, LIF Neurons, and STDP learning rules. Definition of Scope: MNIST Classification using Brian2. Environment Setup: Brian2, NumPy, Git repo initialization.
II. Implementation	Weeks 4–8 (Nov 5 – Dec 8)	Data Encoding: Implementing Poisson spike generation for MNIST. Model Building: Coding SNN with NeuronGroup and STDP Synapses. Implementation of Winner-Take-All (WTA) inhibition. Milestone M2: Functional SNN model built.
III. Experimentation	Weeks 9–11 (Dec 9 – Dec 29)	Training & Tuning: Hyperparameter optimization (learning rates, time constants). Evaluation on test set and generation of raster plots/confusion matrices. Milestone M3: Results finalized for reporting.
IV. Finalization	Weeks 12–15 (Dec 30 – Jan 23)	Stretch Goal: Comparative trials with snnTorch (if time permits). Code cleanup and final report writing. Milestone M5: Final Submission (Jan 23).

TABLE IV: Methodological Deviations from Initial Proposal

Component	Initial Proposal	Final Implementation & Justification
Dataset	Standard MNIST (Static Images)	N-MNIST (Neuromorphic Event Stream) Justification: Static MNIST images converted to Poisson spikes do not capture the true temporal dynamics. N-MNIST was chosen to validate temporal sparsity on real event data.
Learning Rule	STDP (Unsupervised) using Brian2	Surrogate Gradients (Supervised) using snnTorch Justification: While biologically plausible, STDP proved difficult to scale for deep convolutional architectures. The training of model took several hours. Hence, to refer with the provided research papers and to achieve competitive accuracy, the project shifted to BPTT with surrogate gradients using snnTorch.

Project Workload and Timeline Distribution

The project was executed over a comprehensive 15-week timeline, structured to fulfill the academic requirements of a 7 ECTS Master's module which corresponds to a total workload of approximately 210 hours. This time budget was rigorously allocated across four distinct phases to ensure the successful delivery of both the neuromorphic implementation and the final IEEE-standard research paper.

Phase I: Research & Setup (Weeks 1–3 $|\approx$ 40 Hours)

The initial phase was dedicated to establishing a robust theoretical framework, accounting for approximately 40 hours of the total workload. A significant portion of this time was invested in a deep-dive literature review focusing on the differential equations governing Leaky Integrate-and-Fire (LIF) neuron dynamics. The mathematical foundations of synaptic plasticity, specifically Spike-Timing-Dependent Plasticity (STDP), were rigorously analyzed to understand their biological plausibility versus computational feasibility. Concurrently, the development environment was initialized, involving the installation and configuration of the Brian2 simulator and NumPy libraries to ensure compatibility with Python-based workflows. Early validation of the project scope was performed by mathematically verifying the theoretical limits of unsupervised learning rules on simple datasets. This phase also included the setup of a version-controlled Git repository to track algorithmic changes and ensure reproducibility from day one. By the end of Week 3, a solid theoretical roadmap was established, defining the mathematical boundaries for the subsequent implementation phase.

Phase II: Implementation (Weeks 4–8 $|\approx$ 80 Hours)

This phase proved to be the most labor-intensive and critical portion of the project, consuming roughly 80 hours due to unforeseen technical challenges and necessary strategic pivots. Initially, the project aimed to implement an unsupervised SNN using the Brian2 framework; however, early tests revealed that STDP learning rules struggled to converge on complex convolutional architectures. This realization necessitated a significant pivot in the learning strategy, shifting from unsupervised biological rules to Supervised Surrogate Gradient learning. This strategic shift was directly influenced by the foundational frameworks established in the literature. Specifically, the "SuperSpike" methodology proposed by [7] was adopted to derive the Fast Sigmoid surrogate derivative, allowing the model to overcome the non-differentiable nature of spikes as mentioned in Section II (5). Furthermore, the learning rule was re-architected to utilize Backpropagation Through Time (BPTT) as detailed by [3], enabling the effective optimization of deep spiking architectures which STDP failed to support. Simultaneously, a second pivot was required regarding the dataset; the conversion of static MNIST images into Poisson spikes failed to capture genuine temporal dynamics, forcing a transition to the event-based N-MNIST dataset [5]. These simultaneous pivots required a complete refactoring of the data ingestion pipeline and the adoption of the snnTorch

framework [8], which added approximately 25 to 30 hours of unexpected study and coding time. The implementation of the Fast Sigmoid surrogate gradient function was mathematically verified against the derivative of the Heaviside step function to ensure gradient flow during backpropagation. Furthermore, the codebase was expanded to include custom data loaders capable of binning asynchronous events into discrete time steps for the new architecture. This period was defined by an iterative cycle of coding, mathematical debugging, and re-implementation to align the software with modern neuromorphic standards.

Phase III: Experimentation (Weeks 9–11 $|\approx$ 50 Hours)

Approximately 50 hours were allocated to the rigorous testing, hyperparameter optimization, and validation of the comparative models. This phase involved a systematic search for optimal SNN parameters, specifically tuning the membrane decay rate (β) and the surrogate gradient slope (k) to balance temporal memory with gradient stability. A critical component of this phase was the validation of the sparsity metrics, where custom evaluation scripts were written to mathematically verify that the network remained silent for the majority of time steps ($> 96\%$). The experimental design was strictly controlled, requiring the training of a structurally identical ANN baseline to ensure a fair comparison of algorithmic efficiency. Theoretical energy consumption was calculated using standard operation-counting benchmarks (MACs vs. ACs), requiring precise tracking of spike counts across all layers during inference. Troubleshooting convergence issues on the N-MNIST dataset required multiple retraining sessions to isolate backend-specific numerical instabilities between CUDA and MPS. The generation of final visual evidence, including spike raster plots and confusion matrices, was also done during this phase to support the findings.

Phase IV: Finalization (Weeks 12–15 $|\approx$ 40 Hours)

The final 4-week block focused on compiling the experimental data into a scientific narrative, consuming about 40 hours of effort. This phase began with a comprehensive code cleanup and refactoring process to ensure the reproducibility of results and the documentation. The writing of the final IEEE-standard report required the careful integration of mathematical formulations, specifically the description of the BPTT framework and the energy estimation logic. Each scientific claim made in the paper was cross-verified against the experimental logs to ensure absolute accuracy in the reported accuracy and sparsity figures. The ”Statement of Generative AI Usage” was also drafted and refined during this period to ensure full transparency regarding tool utilization. Additionally, the project’s visual assets, such as the comparative architecture diagrams and timeline tables, were finalized to meet academic presentation standards. The phase concluded with a final rigorous review of the bibliography and the successful submission of the Milestone M5 deliverables.